Replace Better Auth with Custom JWT Authentication

Overview

The project currently uses **Better Auth** — a third-party library — for all authentication. The requirement is to replace this with a **fully custom-built JWT implementation** that lives entirely in our own code. The overhaul touches the DB schema, every backend route, and the entire frontend auth layer.

Key Design Decisions

> [!IMPORTANT]
> **Two-layer authorization model** — There is a *system-level* role (`ADMIN` / `USER` in the `users` table) AND an *election-scoped* role (`RO` / `PRO` / `PO` in `role_map`). Both must be enforced independently at the backend.

> [!IMPORTANT]
> **Approval gate** — A user with `approved = false` cannot log in at all, even with the correct password. The login endpoint checks `approved` before issuing any JWT.

> [!IMPORTANT]
> **JWT (not session IDs)** — We'll use signed JWTs stored in `httpOnly` cookies. This satisfies "your own code" because we generate/verify them ourselves with `jsonwebtoken`, without any third-party auth service.

> [!IMPORTANT]
> **DB schema is being replaced** — The Better Auth tables (`account`, `session`, `verification`) will be **dropped**. The `"user"` table will be **replaced** with a clean `users` table (lowercase, integer PK). The `role_map` table references `user_id`; all cascades are set up so deleting a user cleans dependent rows automatically.

> [!WARNING]
> **All existing data will be wiped** — The migration script will `DROP` old BA tables and truncate the `users` table before re-seeding. Non-auth tables (election, constituency, polling_center, etc.) are preserved in structure but role_map rows that reference old user IDs will be cleaned via CASCADE.

> [!NOTE]
> **ADMIN role stays in the `users` table** — The `role` column on `users` holds either `'ADMIN'` or `'USER'`. The fine-grained worker role (RO / PRO / PO) is in `role_map` and stays there. The JWT payload carries the system role only; election-scoped role checks are done by querying `role_map` at the backend endpoint level.

Proposed Changes

1. Database — Schema Migration

Run a migration SQL script that:
1. Drops Better Auth tables: `DROP TABLE IF EXISTS verification, session, account CASCADE`
2. Drops the old `"user"` table (camelCase columns) with CASCADE
3. Creates a clean `users` table:
   ```sql
   ```

```
   CREATE TABLE users (
     id          SERIAL PRIMARY KEY,
     name        TEXT NOT NULL,
     email       TEXT NOT NULL UNIQUE,
     password    TEXT NOT NULL,          -- bcrypt hash
     role        TEXT NOT NULL DEFAULT 'USER',  -- 'ADMIN' or 'USER'
     approved    BOOLEAN NOT NULL DEFAULT false,
     created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
   );
   ```
4. Updates `role_map` FK: `user_id INTEGER REFERENCES users(id) ON DELETE CASCADE`

2. Backend — New Packages

Install: `bcryptjs`, `jsonwebtoken`, `cookie-parser`
Remove: `better-auth`

[MODIFY] `package.json`
Add `bcryptjs`, `jsonwebtoken`, `cookie-parser`; remove `better-auth`.

[DELETE] `auth.js` (root)
The Better Auth config file is entirely removed.

[MODIFY] `index.js`
- Remove `better-auth/node` import and `toNodeHandler` mount
- Add `cookie-parser` middleware
- Add new auth routes: `app.use('/api/auth', require('./routes/auth'))`

[NEW] `routes/auth.js`
Custom auth endpoints:
- `POST /api/auth/login` — verify email/password with bcrypt, issue JWT in `httpOnly` cookie
- `POST /api/auth/logout` — clear the cookie
- `GET /api/auth/me` — return the current user from the JWT (used by frontend `check` / `getIdentity`)

The JWT payload:
```json
{
  "sub": 1,
  "name": "Admin User",
  "email": "admin@election.dev",
  "role": "ADMIN",
  "approved": true,
  "iat": ...,
  "exp": ...
}
```

[MODIFY] `middleware/auth.js`
Completely rewritten to:
- Read JWT from `req.cookies.token`
- Verify with `jsonwebtoken` using `JWT_SECRET`
- Attach `req.user = { id, name, email, role, approved }` on success
- `requireRole(...roles)` works identically to before

[MODIFY] `routes/signup.js`
Remove Better Auth API call. Use `bcryptjs.hash()` directly, then `INSERT INTO users (name, email, password, role, approved) VALUES (...)`.

[MODIFY] `routes/admin.js`
- `reject` endpoint: no longer needs to delete from `session`/`account` — just `DELETE FROM users WHERE id = $1`

[MODIFY] `routes/users.js`
- `/ro`, `/pro`, `/po`, `/assignable`, `/my-elections` query `public.users` (new table name, integer ids)
- `/my-elections` reads `req.user.id` from the JWT instead of a query param — now a **protected** route requiring `requireAuth`

---

4. Backend — Election-Scoped Role Authorization

This is the core of the new RBAC work. A USER with RO role in election1 must NOT be able to mutate or read protected data belonging to election2, or impersonate a PRO/PO role they don't hold.

How it works

We add a new middleware factory: `requireElectionRole(role, paramName)`.

```js
// middleware/auth.js (addition)
async function requireElectionRole(role, relationParam) {
  return async (req, res, next) => {
    const userId = req.user.id;
    const relationId = req.params[relationParam];
    const result = await pool.query(
      `SELECT 1 FROM role_map
       WHERE user_id = $1 AND role = $2 AND relation_id = $3`,
      [userId, role, relationId]
    );
    if (result.rowCount === 0) return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}
```

This middleware is applied to the mutation/read endpoints that are specific to a user's assignment:

Endpoint	Guard
`GET /api/polling_center_of_election/election/:eId/constituency/:cId`	`requireAuth` + verify user has RO role for a `coe` in that election
`POST /api/polling_center_of_election`	`requireAuth` + user must be RO for the relevant `coe`
`PUT /api/polling_center_of_election/:poeId/pro`	`requireAuth` + user must be RO whose `relation_id` covers the given `poeId`
`DELETE /api/polling_center_of_election/:poeId`	same as above
`GET/POST/DELETE /api/candidate/coe/:coeId`	`requireAuth` + user must be RO with `relation_id = coeId`
`GET /api/candidate/voter-count/:coeId`	same
`GET /api/candidate/center-voter-counts/:coeId`	same
`GET /api/polling_booth /booth endpoints` (PRO)	`requireAuth` + user must be PRO with `relation_id = poeId`
`GET /api/voters / voter-allocation` (PRO or PO)	`requireAuth` + scoped check

for PRO or PO assignment |
| `GET /api/kiosk/*` | public (kiosk doesn't require login) |
| `GET/POST/PUT/DELETE /api/election/*` | `requireAuth` + `requireRole('ADMIN')` |
| `GET/POST/PUT/DELETE /api/constituency/*` | `requireAuth` +
`requireRole('ADMIN')` |

What this prevents

- **URL spoofing:** A user who IS an RO for election1 tries to call `PUT /api/polling_center_of_election/999/pro` where `poeId=999` belongs to election2. The middleware looks up `role_map WHERE user_id=X AND role='RO' AND relation_id covers poeId 999` — this returns 0 rows → **403 Forbidden**.
- **Role spoofing:** A user who is a PO tries to call an RO-only endpoint. `requireElectionRole('RO', ...)` returns 0 rows for them → **403**.
- **Cross-election access:** Same user, different election — role_map is election-scoped, so the query still returns 0 rows.

Frontend note (defense-in-depth)

The frontend already provides a first line of defense: `ElectionEntry.tsx` calls `/api/users/my-elections` (protected, returns only the JWT owner's assignments), finds the assignment for the election, and renders _only_ the correct role component (RODashboard / PRODashboard / PODashboard). A user who types `/homeUSER/election/1` directly will only ever see their own dashboard for that election — or an error if they have no assignment.

However, because the dashboards call backend APIs directly using `const API = 'http://localhost:3001/api'` + plain `fetch()`, we **cannot** rely solely on the frontend. Backend enforcement via `requireElectionRole` is the real security boundary.

---

5. Approval System

The `approved` column is preserved in the new `users` table. The system works as follows:

Login flow (with approval gate)
```
1. POST /api/auth/login { email, password }
2. Find user by email → if not found → 401
3. bcrypt.compare(password, user.password) → if wrong → 401
4. if user.approved === false → 403 { error: 'Account pending approval' }
5. Issue JWT, set httpOnly cookie
```

This means an unapproved user gets a clear "pending approval" error at login, exactly as before.

Admin approval/rejection
- `GET /api/admin/pending` → lists all users where `approved = false` (guarded by `requireAuth + requireRole('ADMIN')`)
- `POST /api/admin/approve/:userId` → sets `approved = true`
- `POST /api/admin/reject/:userId` → `DELETE FROM users WHERE id = $1` (cascades to role_map, no separate session/account cleanup needed)

The existing `pendingApprovals.tsx` frontend page works unchanged — it already uses `credentials: 'include'` fetch calls, which will automatically send the JWT cookie.

[MODIFY] All other route files (`adminHome.js`, `constituency.js`, etc.)
Currently **no auth middleware** is applied to most routes. We will add `requireAuth` (and `requireRole('ADMIN')` where appropriate) to all protected endpoints as part of this work.

[MODIFY] `seed.js`
Rewritten to use bcryptjs directly (no Better Auth API). Seeds 3 users: admin, an RO-role user, a PO-role user.

[MODIFY] `.env`
Replace `BETTER_AUTH_SECRET` / `BETTER_AUTH_URL` with:
```
JWT_SECRET=<strong-random-secret>
JWT_EXPIRES_IN=8h
```

---

3. Frontend

[MODIFY] `src/lib/auth-client.ts`
Replace the Better Auth client with a thin custom `apiClient` utility:
```ts
// Simple fetch wrapper that always sends credentials (cookie)
export const apiClient = {
  get: (path) => fetch(`http://localhost:3001${path}`, { credentials: 'include' }),
  post: (path, body) => fetch(`http://localhost:3001${path}`, { method:'POST', credentials:'include', headers:{'Content-Type':'application/json'}, body: JSON.stringify(body) }),
};
```

[MODIFY] `src/providers/auth-provider.ts`
Completely rewritten. No more `authClient.*` calls. Uses `apiClient`:
- `login` → `POST /api/auth/login` → reads role from response JSON → redirects
- `logout` → `POST /api/auth/logout`
- `check` → `GET /api/auth/me` → 200 = authenticated
- `getIdentity` → `GET /api/auth/me` → returns user object
- `getPermissions` → `GET /api/auth/me` → returns role

[MODIFY] `src/pages/signup.tsx`
No changes needed — it already calls `POST /api/signup` directly with `fetch`, which we're rewriting on the backend.

[MODIFY] `src/pages/admin/pendingApprovals.tsx`
The `credentials: 'include'` fetch calls work as-is since auth is now cookie-based JWT. **No frontend change needed** — the backend now correctly reads the cookie.

[MODIFY] `src/pages/user/userDashboard.tsx` & `ElectionEntry.tsx`
Currently pass `?userId=...` as a query param to `/api/users/my-elections`. After the overhaul, the backend reads `req.user.id` from the JWT, so the frontend just calls `/api/users/my-elections` with `credentials: 'include'` — **no userId query param needed**.

[MODIFY] `src/pages/user/election/RODashboard.tsx`, `PRODashboard.tsx`, `PODashboard.tsx`, `VoterAllocationTab.tsx`
All dashboards use `const API = 'http://localhost:3001/api'` and call it with plain `fetch(url)` — no `credentials: 'include'`. This must change: **every backend call

from these dashboard files needs `credentials: 'include'`** so the JWT cookie is sent and the backend can verify identity and run `requireElectionRole` checks. A shared `apiFetch(url, options)` helper will be added to `src/lib/auth-client.ts` that always injects `credentials: 'include'`.

Implementation Order

1. Install new backend packages (`bcryptjs`, `jsonwebtoken`, `cookie-parser`), uninstall `better-auth`
2. Write and run DB migration SQL (drop BA tables, create new `users` table)
3. Rewrite `middleware/auth.js` (JWT verify + `requireRole` + `requireElectionRole`)
4. Create `routes/auth.js` (login with approval gate, logout, /me)
5. Rewrite `routes/signup.js` (bcryptjs directly, no BA)
6. Update `routes/admin.js` (approve/reject against new `users` table)
7. Update `routes/users.js` (new table name, protect `/my-elections`)
8. Add `requireAuth` + scoped guards to all protected route files
9. Update `index.js` (remove BA handler, add cookie-parser, mount auth route)
10. Rewrite `seed.js` and run it
11. Rewrite frontend `src/lib/auth-client.ts` (add `apiFetch` helper with credentials)
12. Rewrite frontend `src/providers/auth-provider.ts`
13. Update `userDashboard.tsx` and `ElectionEntry.tsx` (remove `?userId=...`, use `apiFetch`)
14. Update all dashboard files to use `apiFetch` instead of bare `fetch`
15. Test login flow: admin, approved USER, unapproved USER
16. Test role enforcement: RO on election1 cannot access PODashboard or RO data on election2

Verification Plan

Automated
- Run `node seed.js` — confirms DB migration + user creation succeed
- `curl POST /api/auth/login` with admin credentials — confirm cookie is set and contains valid JWT
- `curl GET /api/auth/me` with cookie — confirm user data returned
- `curl GET /api/admin/pending` without cookie — confirm 401

Manual (Browser)
- Navigate to `/login`, log in as admin → lands on `/homeAdmin`
- Navigate to `/login`, log in as worker → lands on `/homeUSER`
- Try accessing `/homeAdmin` as a USER → redirected to `/homeUSER`
- Unapproved user login → friendly error message
- Sign up as new user → pending approval; can't log in until approved